МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ

Лабораторная работа № 3

по дисциплине «Математические основы баз данных»

по теме: «Многотабличные запросы и аналитические представления»

Выполнил студент группы:
БВТ2505
Соронина Арина Андреевна
Проверил: Быватов Семён Юрьевич

Москва 2026

Цель и задачи работы:

Цель: составить набор многотабличных и аналитических запросов к базе данных кинотеатра и оформить наиболее полезные из них в виде представлений.

Задачи:

1. Напишите запрос с внутренним соединением не менее трёх таблиц

2. Напишите запрос с LEFT JOIN, показывающий строки без соответствий в связанной таблице.

3. Постройте запрос с группировкой и агрегацией.

4. Примените HAVING для фильтрации групп.

5. Напишите по одному запросу с подзапросом каждого вида: скалярным, с IN и с EXISTS (или коррелированным).

6. Используйте CTE для разбиения сложного запроса на шаги; реализуйте рекурсивный CTE для обхода дерева категорий.

7. Создайте не менее двух представлений для типовых отчётов и продемонстрируйте обращение к ним.

Выполнение

1. *Запрос с внутренним соединением (INNER JOIN) не менее трёх таблиц*

Связываем фильмы (movies), их жанры (genres) и прокатчиков (distributors)

```
SELECT
       m.title AS movie_title,
       g.name AS genre_name,
       d.company_name AS distributor_name
FROM movies m
INNER JOIN genres g ON m.genre_id = g.genre_id
INNER JOIN distributors d ON m.distributor_id = d.distributor_id;
```

	movie_title character varying (200)	genre_name character varying (100)	distributor_name character varying (150)
1	Битва потерянных душ 4	Мелодрама	Toho
2	Хозяин третьей планеты 4	Фантастика	Toho
3	Тайна третьей планеты 4	Триллер	Централ Партнершип
4	Зов ночного патруля 3	Фэнтези	Централ Партнершип
5	Зов третьей планеты 2	Мелодрама	Constantin Film

2. *Запрос с LEFT JOIN для поиска строк без соответствий.*

Находим клиентов (customers), которые еще не сделали ни одного бронирования (bookings)

```
SELECT
    c.full_name,
    c.email
FROM customers c
LEFT JOIN booking b ON c.customer_id = b.customer_id
WHERE b.booking_id IS NULL;
```

	full_name character varying (150)	email character varying (150)
1	Козлов Сергей	[null]
2	Новиков Дмитрий	user887@example.com
3	Попова Ольга	user599@example.com
4	Михайлов Сергей	user151@example.com
5	Соколова Мария	user399@example.com
6	Петров Иван	user119@example.com

3. *Запрос с группировкой и агрегацией.*

Рассчитываем количество фильмов и их среднюю продолжительность в разрезе каждого жанра.

```
SELECT
    g.name AS genre_name,
    COUNT(m.movie_id) AS movies_count,
    ROUND(AVG(m.duration_min), 1) AS avg_duration
FROM genres g
LEFT JOIN movies m ON g.genre_id = m.genre_id
GROUP BY g.genre_id, g.name;
```

	genre_name character varying (100)	movies_count bigint	avg_duration numeric
1	Мелодрама	110	126.2
2	Драма	106	131.9
3	Комедия	113	134.2
4	Триллер	91	131.0
5	Фэнтези	94	132.7
6	Фантастика	99	132.1

4. *Использование HAVING для фильтрации групп.*

Отбираем клиентов (customers), суммарный объем платежей (payments) которых по всем бронированиям превышает 5000.

```
SELECT
    c.full_name,
    SUM(p.amount) AS total_spent
FROM CUSTOMERS c
JOIN bookings b ON c.customer_id = b.customer_id
JOIN payments p ON b.booking_id = p.booking_id
GROUP BY c.customer_id, c.full_name
HAVING SUM(p.amount) > 5000;
```

	full_name character varying (150)	total_spent numeric
1	Фёдоров Сергей	6100.00
2	Иванов Алексей	5900.00
3	Смирнов Андрей	6750.00
4	Новикова Елена	8200.00
5	Морозова Елена	5900.00

5. *Скалярный подзапрос.*

Находим фильмы с продолжительностью выше средней по базе.

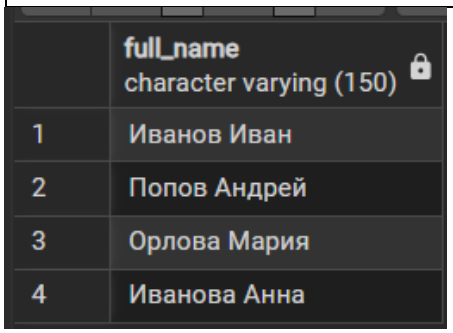
```
SELECT title, duration_min
FROM movies
WHERE duration_min > (SELECT AVG(duration_min) FROM movies);
```

	title character varying (200)	duration_min integer
1	Битва потерянных душ 4	135
2	Хозяин третьей планеты 4	158
3	Охота северных гор 3	155
4	Побег ночного патруля	141
5	Путь ночного патруля 4	173

6. *Подзапрос с IN.*

Получаем список актёров, игравших в фильме с movie_id = 1 через таблицу связей movie_cast.

```
SELECT full_name
FROM actors
WHERE actor_id IN (
    SELECT actor_id
    FROM movie_cast
    WHERE movie_id = 1
);
```

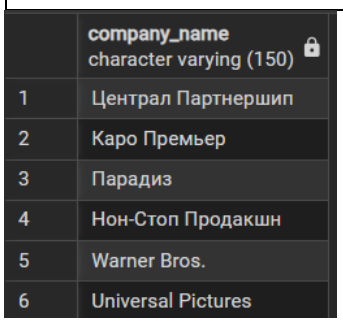


	full_name character varying (150)
1	Иванов Иван
2	Попов Андрей
3	Орлова Мария
4	Иванова Анна

7. *Подзапрос с EXISTS.*

Вывести прокатчиков (distributors), фильмы которых присутствуют в прокате.

```
SELECT company_name
FROM distributors d
WHERE EXISTS (
    SELECT 1
    FROM movies m
    WHERE m.distributor_id = d.distributor_id
);
```



	company_name character varying (150)
1	Централ Партнершип
2	Каро Премьер
3	Парадиз
4	Нон-Стоп Продакшн
5	Warner Bros.
6	Universal Pictures

Использование CTE.

Агрегация выручки по каждому сеансу

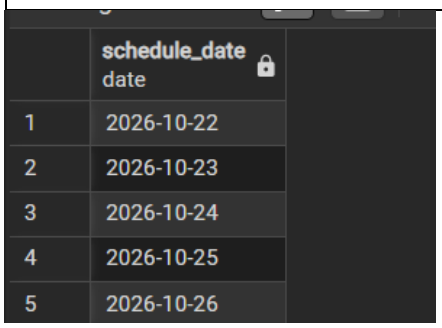
```
WITH SessionRevenue AS (
    SELECT
        session_id,
        SUM(price) AS total_revenue
    FROM tickets
    GROUP BY session_id
)
SELECT
    s.start_time,
    sr.total_revenue
FROM sessions s
JOIN SessionRevenue sr ON s.session_id = sr.session_id;
```

	start_time timestamp without time zone	total_revenue numeric
1	2026-09-01 10:00:00	2600.00
2	2026-09-01 10:00:00	2800.00
3	2026-09-01 10:00:00	750.00
4	2026-09-01 13:00:00	3000.00
5	2026-09-01 13:00:00	800.00

8. *Использование рекурсивного CTE.*

Генерация дат на неделю вперед для сетки сеансов

```
WITH RECURSIVE Calendar AS (
    SELECT CURRENT_DATE AS schedule_date
    UNION ALL
    SELECT schedule_date + 1
    FROM Calendar
    WHERE schedule_date < CURRENT_DATE + 6
)
SELECT schedule_date FROM Calendar;
```



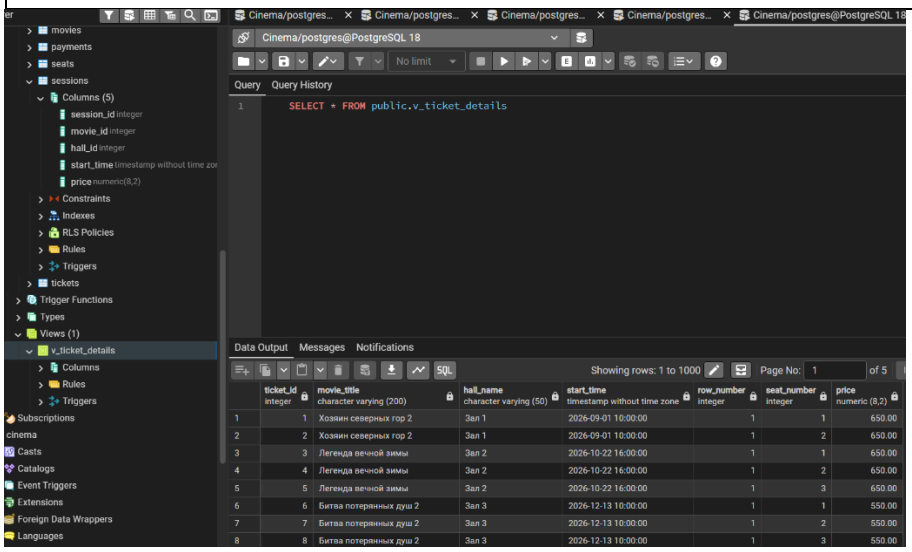
	schedule_date date
1	2026-10-22
2	2026-10-23
3	2026-10-24
4	2026-10-25
5	2026-10-26

9. *Создание представлений (VIEW)*

Представление 1: Детальная информация по билетам (соединение 5 таблиц для полной картины места и сеанса).

```
CREATE VIEW v_ticket_details AS
SELECT
    t.ticket_id,
    m.title AS movie_title,
    h.name AS hall_name,
    s.start_time,
    st.row_number,
    st.seat_number,
    t.price
FROM tickets t
JOIN sessions s ON t.session_id = s.session_id
JOIN movies m ON s.movie_id = m.movie_id
JOIN halls h ON s.hall_id = h.hall_id
```

```
JOIN seats st ON t.seat_id = st.seat_id;
```



Представление 2: Статус бронирований и оплат клиентов.

```
CREATE VIEW v_customer_bookings AS

SELECT

    b.booking_id,

    c.full_name,

    c.phone,

    b.booking_date,

    b.status AS booking_status,

    p.amount AS payment_amount,

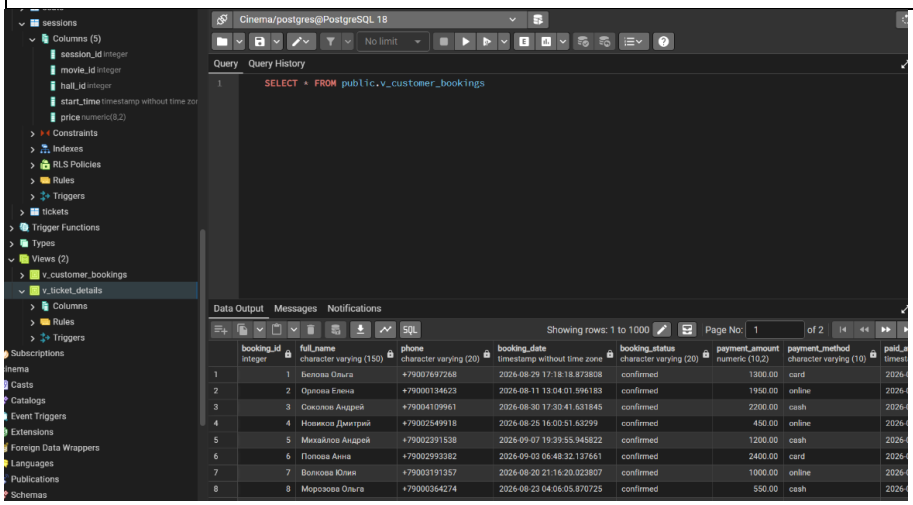
    p.payment_method,

    p.paid_at

FROM bookings b

JOIN customers c ON b.customer_id = c.customer_id

LEFT JOIN payments p ON b.booking_id = p.booking_id;
```



Вывод

В ходе выполнения работы были освоены и применены на практике ключевые инструменты SQL в СУБД PostgreSQL. Были построены многотабличные запросы с использованием `INNER JOIN` и `LEFT JOIN` для объединения данных и выявления отсутствующих связей, а также реализована аналитическая группировка с фильтрацией агрегатов через `GROUP BY` и `HAVING`. Для комплексного отбора данных задействованы скалярные и предикатные подзапросы с `IN` и `EXISTS`. С помощью обобщенных табличных выражений (`CTE`), включая рекурсивный запрос, структурированы вычисления и сгенерирована календарная сетка сеансов, а созданные представления (`VIEW`) инкапсулировали сложную логику соединений в готовые типовые отчёты. Все требования технического задания выполнены в полном объёме.